

Containers and Kubernetes TASK – 3

1. Write a short note (4-5 lines) explaining how Kubernetes orchestration helps apps like Zomato or Swiggy handle thousands of food orders at once without downtime.

Ans : Kubernetes orchestration helps apps like Zomato and Swiggy manage thousands of food orders by automatically distributing workloads across multiple servers.

It can quickly create more containers when order traffic increases and reduce them when demand falls.

If one container or server fails, Kubernetes automatically restarts or shifts the workload to healthy ones.

This ensures the applications remain highly available and responsive, even during peak hours.

2. Create a Kubernetes Deployment YAML file called spotify-playlist-deployment.yaml that runs two containers: one for a playlist API (use nginx:latest as a placeholder) and one for a song database (use mongo:latest).

Ans :

```
# spotify-playlist-deployment.yaml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: spotify-playlist-deployment
```

```
spec:
```

```
  replicas: 2
```

```
  selector:
```

matchLabels:

app: spotify-playlist

template:

metadata:

labels:

app: spotify-playlist

spec:

containers:

- name: playlist-api

image: nginx:latest

ports:

- containerPort: 80

- name: song-database

image: mongo:latest

ports:

- containerPort: 27017

3. Apply your spotify-playlist-deployment.yaml using kubectl and verify that both containers are running by describing the created pods.

Ans :

```
cat > spotify-playlist-deployment.yaml <<'EOF'
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:  
  name: spotify-playlist-deployment
```

```
spec:
```

```
  replicas: 2
```

```
  selector:
```

```
    matchLabels:
```

```
      app: spotify-playlist
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: spotify-playlist
```

```
    spec:
```

```
      containers:
```

```
        - name: playlist-api
```

```
          image: nginx:latest
```

```
          ports:
```

```
            - containerPort: 80
```

```
        - name: song-database
```

```
          image: mongo:latest
```

```
          ports:
```

```
            - containerPort: 27017
```

```
EOF
```

```
kubectl apply -f spotify-playlist-deployment.yaml
```

```
kubectl get deployments
```

```
kubectl get pods -l app=spotify-playlist
```

```
kubectl describe pod $(kubectl get pods -l app=spotify-playlist -o  
jsonpath='{.items[0].metadata.name}')
```

```
kubectl get pods -l app=spotify-playlist
```

**4.Create a Kubernetes Service YAML file called spotify-playlist-service.yaml to expose your playlist API container on port 8080 within the cluster.

Hint: Use type: ClusterIP and targetPort: 80 for the nginx container.**

Ans :

```
cat > spotify-playlist-service.yaml <<'EOF'
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: spotify-playlist-service
```

```
spec:
```

```
  type: ClusterIP
```

```
  selector:
```

```
    app: spotify-playlist
```

```
  ports:
```

```
    - protocol: TCP
```

port: 8080

targetPort: 80

EOF

```
kubectl apply -f spotify-playlist-service.yaml
```

```
kubectl get service spotify-playlist-service
```

```
kubectl describe service spotify-playlist-service
```

**5.Scale your Deployment to 4 replicas using the kubectl scale command and observe how Kubernetes handles the scaling.

Hint: Use kubectl get pods to check the number of running pods after scaling.**

Ans :

```
kubectl scale deployment spotify-playlist-deployment --replicas=4
```

```
kubectl get deployment spotify-playlist-deployment
```

```
kubectl get pods -l app=spotify-playlist
```

```
kubectl get pods -l app=spotify-playlist -w
```